Standard Operating Procedure

Yale School of Public Health
Data Science and Data Equity

The purpose of this document is to outline the contents and ongoing maintenance procedure of the **Resource Navigation Tool** website.

Developer: Shelby Golden, M.S.

Authors:

- Shelby Golden, M.S.

Date Effective: May 21th, 2026

Date Updated: May 21th, 2026

Table of contents

Important Links

- Website
- GitHub Repository
- SharePoint (located in "DSDE - Core/General/Media & Communications/Website/Resource Navigation Tool")

The project SharePoint folder contains additional resources, including running meeting notes, Excel documents tracking the listing queue for approvals and site updates, and links to surveys. This codebase includes local-only files that each user must download and place in their cloned copy of the repository in order to run the website.

Codebase Overview

File Structure

```
resources-page/
  |- _extensions/
    |- parse-latex/
    |- tarleb/
      |- parse-latex/
  |- _includes/
  |- _site/
  |- _styles/
    |- Yale_typeface/
  |- Code/
    |- Listing_About-Us/
    |- Listing_Datasets/
    |- Listing_Resources/
    |- Standard Operating Procedure (SOP)/
      |- SOP Formatting/
  |- Data/
  |- Images/
    |- Datasets/
    |- DSDE Identity/
    |- Other Infographics/
    |- People/
```

```
  |- Resources/
  |- Timelines/
|- KEEP LOCAL/
  |- Archived Pages/
    |- Old Website Structure_05.11.2025/
    |- Page Filtering Options/
  |- DSDE-Visuals/
    |- Stock-Photos/
    |- Wordmark/
      |- PNG/
      |- SVG/
|- Pages/
  |- More-Info-Pages/
    |- Freely-Accessible/
    |- License-Required/
    |- Restricted-Access/
|- renv/
```

> **Note**
>
> Files within the "Keep Local/Archived Pages/" directory are not documented here, as they are not actively utilized in the CI/CD pipeline. They are maintained for archival purposes only.

Environment Reproducibility

This website was created using R (v 4.5.2) in the RStudio IDE (v 2026.01.1+403) and Quarto (v 1.8.27). The `renv()` package (v 1.1.7) is included to reproduce the same coding environment, ensuring all relevant packages and package versions are stored. If you encounter issues running the scripts, please check that the environment is initialized and that you are using the same versions of R, RStudio, and Quarto.

> **Warning**
>
> Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

If this is the first time the repository is being initialized, you may need to initialize the environment. This step will install all packages and versions used in the codebase, ensuring a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/resource-page/"
```

2. Launch the project by opening the `*.Rproj` file in RStudio.
3. In the R console, activate the environment by running the following lines of code:

Command-Line Interface

```
renv::init()      # Initialize the project.
renv::restore()   # Download packages and their version saved in the
↪  lockfile.
```

> **Note**
>
> If you are asked to update packages, say no. The `renv()` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:
>
> **RStudio Console**
>
> ```
> - The library is already synchronized with the lockfile.
> ```
>
> If you experience any trouble with this step, you might want to confirm that you are using R (v 4.5.2) in the RStudio IDE (v 2026.01.1+403) and Quarto (v 1.8.27). You can also read more about `renv()` in their vignette [1].

Ignored Files

Some project documents, scripts, and files are not distributed on GitHub or tracked by Git. Many of these are stored in `**/KEEP LOCAL/` directories or are explicitly listed under the "Project-Specific Exclusions" section of the `.gitignore` file.

Some of these files are available on the project SharePoint, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the "Codebase Local Only Files" directory. This directory will contain a `README.rtf` file specifying the intended filepath for each file.

The YaleNew typeface must be downloaded directly by the user and requires NetID access: Yale Typefaces. This typeface is only used by `SOP.qmd`. Place the downloaded files in `_styles/`.

Content Curation

Curation lists are maintained in two documents located in the project SharePoint: "Approval List.xlsx" and "Webpage Queue and Status.xlsx". Both documents contain the same types

of information for each curation entry, including required and optional metadata used on the webpage. Items in the "Approval List.xlsx" are queued for committee review and are not guaranteed to be added to the website. Items in the "Webpage Queue and Status.xlsx" are either already published on the website or are queued for addition.

There is no formal process for determining whether a new curation entry should be added to "Approval List.xlsx" first, requiring additional review and approval before being published to the website, or directly to "Webpage Queue and Status.xlsx", indicating it is already approved for inclusion. When in doubt, consult Bhramar and default to adding the entry to "Approval List.xlsx".

Resource Curation

Resources represent Yale University-wide organizations or services that provide some form of data-related service, including analytical tools, consulting, datasets, data-related educational materials, or similar offerings.

1. Identify a new candidate Resource that is not already listed in the "Resources" sheet of "Webpage Queue and Status.xlsx". Candidates may also be submitted through the suggestion box Qualtrics survey or via direct contact.

2. Fill in all required fields and any relevant optional or extended classifier fields (e.g., Distributor: "Multiple"). Required fields will be highlighted in red if left blank.

   If entering this content into "Approval List.xlsx" first, also complete the approval-related columns B through E. All subsequent columns are labeled according to their relevance to either the Dataset or Resources curation type. Required fields will highlight red based on the curation type indicated in column C.

Dataset Curation

Datasets represent data collections created by Yale affiliates or external sources that are relevant and influential to public health and epidemiological research.

1. Identify a new candidate Datasets that is not already listed in the "Datasets" sheet of "Webpage Queue and Status.xlsx". Candidates may also be submitted through the suggestion box Qualtrics survey or via direct contact.

2. Fill in all required fields and any relevant optional or extended classifier fields (e.g., Distributor: "Multiple"). Required fields will be highlighted in red if left blank.

   If entering this content into "Approval List.xlsx" first, also complete the approval-related columns B through E. All subsequent columns are labeled according to their relevance to either the Dataset or Resources curation type. Required fields will highlight red based on the curation type indicated in column C.

Committee Review of the Curation List

No procedure is defined.

Maintaining the Website

Page content is managed through the corresponding `*.qmd` files. Refer to existing files for guidance on content organization and formatting conventions.

Listing Pages

Adding Tiles

Three pages utilize the Quarto Listing feature: the Resources and Datasets filter/search pages, and the People section of the About Us page. Each of these pages or sections is comprised of the following files:

- `*.qmd` — Renders the page on the website.
- `*.yaml` — Contains all information fields and organizes them within defined hierarchies and classifications.
- `*.ejs` — Ingests the `*.yaml` file and processes it through embedded HTML and JavaScript, formatting the tiles and defining page behaviors such as content-responsive text clamping and the "Show More" button.
- `*.scss` — Defines custom styling and some responsive behaviors.

The `*.qmd` files are located with in the "Pages/" directory, the `*.yaml` and and `*.ejs` are in

"Code/Listing_*/" respective directory, and the `*.scss` file is in the "_styles/" directory. Users only need to edit the `*.yaml` to remove, add, or update existing content fields.

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/resource-page/"
```

2. Run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
quarto preview
```

3. Launch the project by opening the `*.Rproj` file in RStudio.
4. Open the `*.yaml` and `*.qmd` files associated with the Listing you want to revise.
5. The header of each `*.yaml` file describes the required and optional fields used by the respective `*.ejs` file, as well as any additional hierarchies and classifications needed to organize the content. Refer to preexisting entries to help determine formatting and structural conventions.
6. To refresh the local preview and reflect changes saved in the `*.yaml`, save a minor change in the associated `*.qmd` file. Be sure to revert these changes once you have finished updating the `*.yaml`.
7. Update the page-level "Last Updated" field in the header section of the `*.yaml` file.
8. For Resources and Datasets, update the "Added to the Website" field in the respective sheet of "Webpage Queue and Status.xlsx".

9. The content is now ready for publishing. This process is described below.

Formatting Banner Images

Images used on the Resources and Datasets tiles must be formatted to ensure they render correctly within the banner section. Use "KEEP LOCAL/Tile Image Formatting.pptx" to standardize image sizes and expand the background with a matching color as needed. Retain slides for previously formatted tiles for easier adjustments.

1. Search for a high-resolution version of the provider logo online and save it to "Images/[Resources|Datasets]/". If no image is found, use a DSDE Identity image from the Yale School of Public Health Communications office; some have been copied to "Images/DSDE Identity/".
2. Open "KEEP LOCAL/Tile Image Formatting.pptx" and duplicate an existing slide to format the tile image.
3. Center the logo within the black box on the slide, which marks the boundaries of the formatted tile image. If needed, fill the box with the background image color for a clean result. Refer to the Yale Libraries, DISSC, and other Yale center slides for additional customization inspiration.
4. Enter presentation mode and screenshot within the black box boundaries, excluding the black border.
5. Save the image to "Images/[Resources|Datasets]/".

> **!** Retain the Raw Image
>
> Retain a copy of the raw logo or background image alongside the formatted version. If a DSDE Identity image is cropped, retain the original and save the cropped version as "*_cropped.[jpeg|png]".

6. Add the image file path to the respective listing `*.yaml` file and save.

7. **OPTIONAL:** Preview the changes using a local preview. Navigate to the codebase directory in the command-line application (Terminal for Mac or Git Bash for Windows).

Command-Line Interface

```
cd "FILE-PATH/resource-page/"
```

8. Run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
quarto preview
```

9. To refresh the local preview and reflect changes saved in the `*.yaml`, save a minor change in the associated `*.qmd` file. Be sure to revert these changes once you have finished updating the `*.yaml`.

More Info Pages

Each Dataset has an associated "More Info" `*.qmd` page that provides additional details about that dataset. Each page contains four sections: Overview, Gaining Access, Relevant Links, and Publications. References are compiled using BibTeX, which automatically renders them at the end of the page.

A custom JavaScript function was implemented to allow users to zoom into images. Use the following example as a guide for incorporating images with the zoom feature.

***.qmd**

````
<div class="container" data-min-scale="1" data-max-scale="3" style="width:
↪  100%;">
  <img id="myImage"
  ↪  src="../../../Images/Timelines/AllofUs_DURA_Infographic_Downloaded06012025.png"
  ↪  alt="Zoomable Image" title="Click to zoom in">
  <figcaption class="caption">
    The DURA application timeline varies, as institutions may experience
    ↪  different lengths of time during the application process. For additional
    ↪  questions, please contact <a
    ↪  href="mailto:aoudurasupport@vumc.org">aoudurasupport@vumc.org</a>
    ↪  [@support].
  </figcaption>
</div>

```=html
<!-- The Modal -->
<div id="myModal" class="modal">
  <span class="close">×</span>
  <img class="modal-content" id="img01">
  <div id="caption"></div>
</div>

<script src="../../../Code/zoom.js"></script>
```
````

Overview

The Overview section summarizes background details about the data provider and dataset, and describes key features such as study design, sample size, highlights of variable types, demographics. Refer to existing Overview sections for guidance on structure and formatting.

Gaining Access

The Gaining Access section describes how Yale affiliates can access the dataset. For datasets with multiple access levels, each level includes a summary of the qualification requirements, typical timeline, and step-by-step access process. Citing the exact source page for each piece of information is recommended to provide a direct link and capture an date accessed.

Use the panel formatting outline below to get started, and refer to existing "More Info" pages for guidance on structure and formatting.

***.qmd**

```
::: .panel-tabset
### Publicly Available

#### Do I Qualify?

#### Typical Timeline

#### Step-by-Step Guide

### Restricted Access

#### Do I Qualify?

#### Typical Timeline

#### Step-by-Step Guide

:::
```

Relevant Links

The Relevant Links section features important pages from the data provider, including ancillary resources such as research communities and workshops, as well as support pages. Include a brief description for each link summarizing what users can expect to find on that page.

Publications

The Publications section utilizes custom functions that search PubMed for publication matches by DOI. These results then get processed and transformed into the formatted HTML seen on the rendered page (hyperlinked titles, bold-highlighted Yale affiliated authors, journal, etc). JavaScript applies pagination that will restrict publication listings to show three at a time.

There are two sections to this code:

a. Pulling and Formatting Citation Matches

The `get_citations()` function retrieves publication details from PubMed using the listed DOIs. Authors whose primary or secondary affiliation matches the listed affiliations will be formatted with bolded names. An override of explicit names is applied for authors whose affiliation is not listed as Yale.

See the example below for guidance on how to populate the `get_citations()` function. Refer to existing Publications sections for guidance on structure and formatting.

***.qmd Embedded R Script**

```
# Fetch and format the citations.
citations <- get_citations (
  doi = c("10.1093/jamia/ocae098", "10.1002/pon.70302"),
  affiliation = c("Yale School of Public Health", "Yale School of Medicine",
  ↪  "Yale University"),
  highlight_names = c("Bhramar Mukherjee")
)
# Condition the output based on presence of articles.
citations_html <- make_pub_list(citations)
if(!any(str_detect(citations$citations, "No publications"))) {
  citation_length <- length(citations$citations)
  citation_date   <- str_match(citations$query, "\\((\\ d{4}):(\\
  ↪  d{4})\\[dp\\]\\)")[,2]
} else {
  citation_length <- 0
  citation_date   <- as.numeric(format(Sys.Date(), "%Y")) -
  ↪  as.numeric(str_extract("No publications found in the last 25 years.",
  ↪  "[0-9]{2}"))
}
```

b. Display with Pagination

The second section presents the formatted citations with pagination. The script used to render this section is provided in full below.

***.qmd**

````
This section presents a selection of PubMed articles that utilize the dataset and
↪  are authored by individuals affiliated with the Yale University. These
↪  articles are provided to inspire researchers and students to use the data in
↪  their own work.

::: style="margin-left: 1rem"

```=r
#| echo: false
#| results: 'asis'

# Display formatted citations.
cat(citations_html)
```

```=html
<!-- Pagination buttons and scripting -->
<div class="citations-navigation">
  <button onclick="prevPage()">◀ Previous</button>
  <button onclick="nextPage()">Next ▶</button>
</div>

<script src="../../../Code/pagination.js"></script>
```

:::
````

References

Citations must be exported in BibTeX format. Load the corresponding *.bib file in the YAML header of each *.qmd page to automatically render the citations at the end of the document

without additional code. Include a `# References` heading at the end of the page to title the section.

About Us

This page does not require regular updates or maintenance. Links and copy can be revised directly in the respective HTML or Markdown sections. To update the People section, refer to the section about about maintaining Listings.

Publishing Updates

A Shell script was created to render the site while preventing unused media from being transferred to the GitHub repository. Follow the steps below to render and publish the website.

1. Ensure all work is saved and committed.

Command-Line Interface

```
git add <"FILE-NAME"|.>
git commit -m "Your message"
git push
```

2. Render the site locally. The configuration updates the `_site` directory, which is excluded from version control by the `.gitignore` file. This process may take a few minutes to complete.

Command-Line Interface

```
quarto render
```

3. Make sure the shell script is executable.

Command-Line Interface

```
chmod +x Code/copy_images.sh
```

4. Run the script.

Command-Line Interface

```
./Code/copy_images.sh
```

5. **OPTIONAL:** Verify that all the correct files have been rendered to the `_site` directory.

Command-Line Interface

```
ls -R _site
```

6. If everything looks correct, publish the site. Follow the prompts by entering "Yes" to proceed with updating the site and entering any required passwords when prompted.

Command-Line Interface

```
quarto publish gh-pages --no-render
```

If Rendering Fails to Complete

The webpage may occasionally fail to publish; however, this is not necessarily a critical error. If this issue is encountered after attempting to publish, any working trees generated during the incomplete process must be cleared before re-executing the render. This is a known Quarto bug that may be resolved in a future update.

For example, if the working tree is called:

`FILE-PATH/.quarto/quarto-publish-worktree-17fef8679f581e08/`

1. List all currently active worktrees in the environment. One will be associated with `quarto publish` and denoted as "prunable."

Command-Line Interface

```
git worktree list

# Example Result
FILE-PATH/  69edfb0 [main]
FILE-PATH/.quarto/quarto-publish-worktree-71502552fdce11b5  0ee5bea
↪  [gh-pages] prunable
```

2. Forcably removed the prunable worktree.

Command-Line Interface

```
git worktree remove --force
↪  "FILE-PATH/.quarto/quarto-publish-worktree-71502552fdce11b5"
```

3. Retry publishing the site. Follow the prompts by entering "Yes" to proceed with updating the site and entering any required passwords when prompted.

Command-Line Interface

```
quarto publish gh-pages --no-render
```

Maintaining the SOP

This Standard Operating Procedure (SOP) uses a preconfigured LaTeX formatting script. The report `*.qmd` and formatting files are located in "Code/Standard Operating Procedure (SOP)/".

If you are updating the document, please add your name to the **Authors** field and update the **Date Updated** field accordingly. The LaTeX formatting script will automatically include the date the document was rendered at the top of the page.

Indented code blocks use Verbatim code environments, which require additional formatting to align the environment and apply code highlighting. Use "Code Highlighting Formatting_Verbatim.R" to prepare code for this environment and one-level indentation.

Once you have finished editing the document render it, move the resulting `*.pdf` to the project's root directory, then commit and push your changes.

References

1. Ushey, K., Wickham, H. & Posit. Introduction to renv.